Theory Concepts: Risk Alert Classifier

Definitions and explanations of the machine-learning concepts used in this project (classification of customers into Low Risk = 0 and High Risk = 1).

1. Logistic Regression

Logistic Regression
A linear model for classification. It computes a weighted sum of the features and passes it through the sigmoid function to obtain the probability that an observation belongs to the positive class.

```
p(y=1 | x) = 1 / (1 + exp( -(b0 + b1*x1 + ... + bk*xk) ))
```

Why it suits classification:

- Outputs a probability between 0 and 1, so the decision threshold can be moved to trade precision against recall.
- Coefficients are interpretable: exp(b) is the odds ratio for a one-unit increase in a feature.
- Fast, stable, and a strong baseline when classes are close to linearly separable.
- Regularisation (L1/L2, strength 1/C) controls over-fitting; features should be standardised first.

2. The confusion matrix and the error types

	Predicted Low (0)	Predicted High (1)
Actual Low (0)	TN True Negative	FP False Positive (Type-I error)
Actual High (1)	FN False Negative (Type-II error)	TP True Positive

Confusion matrix for the risk-prediction problem.

Type-I error (False Positive)
A Low-Risk customer is predicted High Risk. Cost: unnecessary manual review, possible friction or wrongly restricted credit for a good customer.

Type-II error (False Negative)
A High-Risk customer is predicted Low Risk. Cost: the risk goes undetected (default, fraud or loss). In risk prediction this error is normally far more expensive, so the project minimises it.

3. Classification performance metrics

Accuracy alone is insufficient because it hides how the errors are distributed. With 88% Low-Risk customers, predicting "Low Risk" for everyone gives 88% accuracy while detecting no high-risk customer at all.

Metric	Formula	Question it answers
Accuracy	(TP + TN) / (TP + TN + FP + FN)	What share of all predictions is correct? (misleading when imbalanced)
Precision	TP / (TP + FP)	Of the customers flagged High Risk, how many really are?

Metric	Formula	Question it answers
Recall = TPR (sensitivity)	TP / (TP + FN)	Of all truly High-Risk customers, how many did we catch?
Specificity (TNR)	TN / (TN + FP)	Of all Low-Risk customers, how many were correctly left alone?
FPR	FP / (FP + TN) = 1 - specificity	What share of Low-Risk customers is wrongly flagged?
F1-Score	2 * P * R / (P + R)	Harmonic mean of precision and recall; low if either is low.
F-beta (F2)	(1+b^2) P R / (b^2 P + R), b=2	Like F1 but recall counts twice as much; used when FN is costly.

4. ROC curve and AUC

ROC curve
Plots the True Positive Rate (y-axis) against the False Positive Rate (x-axis) for every possible decision threshold, from strict (nothing flagged) to lenient (everything flagged).

AUC (Area Under the ROC Curve)
A single number between 0 and 1. It equals the probability that the model scores a randomly chosen High-Risk customer higher than a randomly chosen Low-Risk one. 0.5 = random guessing, 1.0 = perfect ranking.

- Threshold-independent, so it compares models on ranking quality rather than on one arbitrary cut-off.
- The curve shows the operating points available: a point high and to the left means high recall at low FPR.
- On heavily imbalanced data, also inspect the precision-recall trade-off, since AUC can look optimistic.

5. Imbalanced data

A dataset is imbalanced when one class is far rarer than the other (here High Risk is about 12%). Training minimises overall error, so the model is rewarded for favouring the majority class. The result is high accuracy but low recall for the minority class, biased probabilities and biased tree splits.

Technique	How it works	Pros	Cons
Random Under-Sampling	Randomly removes majority rows until classes are equal.	Fast; forces focus on minority.	Throws away information; can raise false positives.
Random Over-Sampling	Duplicates random minority rows until classes are equal.	No information lost.	Exact copies encourage over-fitting.
SMOTE	Creates synthetic minority points by interpolating between a minority sample and one of its k nearest minority neighbours.	Adds variety instead of copies.	Can blur class overlap; needs scaled numeric features.
ADASYN	Like SMOTE but generates more synthetic samples for minority points that have many majority neighbours (harder regions).	Adapts to difficult areas.	Can amplify noise and outliers.

Technique	How it works	Pros	Cons
Class weights	Penalises minority errors more in the loss (class_weight='balanced').	No data change.	Needs model support; can lower precision.

Rule: resample only the **training** data, after the train-test split. Resampling before splitting leaks synthetic copies of test rows into training and inflates results.

6. Data preparation concepts

Stratified train-test split
Splits the data so each part keeps the same class proportions as the full data (stratify=y). Essential when the minority class is small, otherwise the test set could contain too few positives.

KNN Imputer (multivariate imputation)
Replaces a missing value with the average (or most common) value of the k most similar rows, where similarity is measured across all the other features. It uses relationships between variables, unlike simple mean imputation. Features must be scaled first, and the imputer is fitted on the training set only.

Data leakage
Any use of information from the test set (or from the target) during training or preprocessing, which gives optimistic results. Avoided here by splitting first and fitting scalers/imputers/resamplers on training data only.

7. Tree-based models

Decision Tree
Repeatedly splits the data on the feature and threshold that best separate the classes (Gini impurity or entropy), ending in leaves that predict a class. Easy to explain but, when grown fully, memorises the training set.

Overfitting
The model fits noise in the training data: training performance is excellent but test performance is much worse. Diagnosed by the train-test gap; controlled with max_depth, min_samples_leaf and pruning (ccp_alpha).

Random Forest
An ensemble of many decision trees, each trained on a bootstrap sample and a random subset of features at every split. Predictions are averaged (bagging). Averaging many de-correlated trees reduces variance, so it generalises better than a single tree.

8. Hyperparameter tuning

Method	Description	When to use
Randomized Search CV	Samples a fixed number of random combinations from the parameter space and scores each with cross-validation.	Wide spaces, many parameters; good first pass.

Method	Description	When to use
Grid Search CV	Exhaustively evaluates every combination in a small grid with cross-validation.	Fine-tuning around a promising region found by Randomized Search.

Both use k-fold stratified cross-validation on the training data and a scoring metric that matches the business goal (F2 here, which prioritises recall). Hyperparameters are set before training (e.g. max_depth, n_estimators, C); parameters (coefficients, split points) are learned from data.

9. Choosing a model for a business requirement

The final model is not simply the one with the highest accuracy. For risk alerts the requirement is to **minimise false negatives**, so models are ranked by recall (miss rate = 1 - recall) subject to an acceptable precision, then verified with cross-validation to avoid selecting on a lucky test split. The decision threshold can then be lowered from 0.5 to catch more high-risk customers at the cost of more reviews.